

LAPORAN PRAKTIKUM
PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 13
NODE.JS: PENGENALAN



Oleh:

Naura Aisha Zahira

2311104078

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

A. Dasar Teori

Node.js merupakan sebuah platform runtime yang memungkinkan bahasa pemrograman JavaScript dijalankan di sisi server. Berbeda dengan JavaScript pada umumnya yang hanya berjalan di browser, Node.js memungkinkan JavaScript digunakan untuk membangun aplikasi backend seperti web server, RESTful API, dan aplikasi berbasis jaringan. Node.js pertama kali dikembangkan oleh Ryan Dahl pada tahun 2009 dan dibangun di atas mesin JavaScript V8 milik Google yang dikenal memiliki performa tinggi dalam mengeksekusi kode JavaScript.

Salah satu keunggulan utama Node.js adalah penerapan konsep event-driven dan non-blocking I/O. Model ini memungkinkan Node.js menangani banyak permintaan secara bersamaan tanpa harus menunggu satu proses selesai terlebih dahulu. Berbeda dengan pendekatan tradisional yang bersifat blocking, Node.js menggunakan mekanisme asynchronous sehingga server tetap responsif meskipun menangani banyak klien secara simultan. Hal ini menjadikan Node.js sangat efisien dan cocok digunakan untuk aplikasi web modern yang membutuhkan performa tinggi dan skalabilitas yang baik.

Node.js juga didukung oleh ekosistem package yang sangat besar melalui NPM (Node Package Manager). Dengan NPM, pengembang dapat dengan mudah menginstal dan mengelola berbagai modul atau library yang dibutuhkan dalam pengembangan aplikasi. Selain itu, Node.js menyediakan berbagai modul bawaan seperti http, fs, dan path yang memudahkan pembuatan server, pengelolaan file, dan manipulasi direktori tanpa perlu menggunakan library tambahan.

Dalam praktikum ini, Node.js digunakan untuk membuat server sederhana menggunakan modul bawaan http. Server ini berfungsi untuk menerima permintaan dari klien melalui browser dan memberikan respons berupa teks. Praktikum ini bertujuan untuk memberikan pemahaman dasar mengenai cara kerja Node.js sebagai server-side JavaScript serta konsep dasar pembuatan web server menggunakan Node.js.

B. Tujuan

1. Ketepatan penerapan pembangunan aplikasi web menggunakan NodeJS.
2. Ketepatan penerapan pembangunan aplikasi web menggunakan Framework NodeJS

BAB II

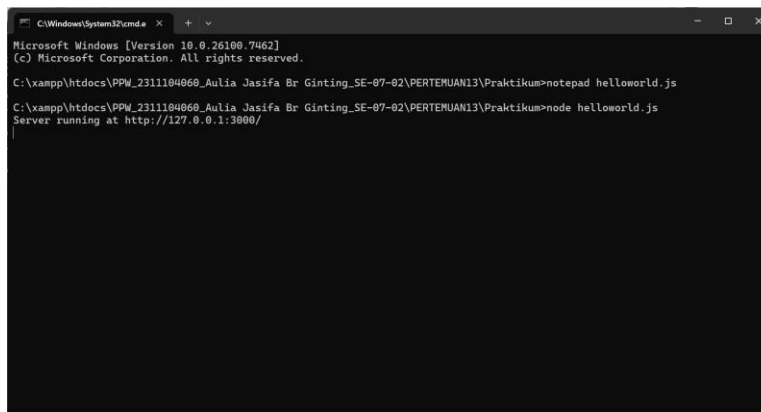
HASIL

A. GUIDED (Praktikum Terbimbing)

a. Contoh node.js

```
JS helloworld.js > ...
1  const { createServer } = require("node:http");
2
3  const hostname = "127.0.0.1";
4  const port = 3000;
5
6  const server = createServer((req, res) => {
7    res.statusCode = 200;
8    res.setHeader("Content-Type", "text/plain");
9    res.end("Hello World");
10 });
11
12 server.listen(port, hostname, () => {
13   console.log(`Server running at http://${hostname}:${port}/`);
14 });
```

Menjalankan code diatas



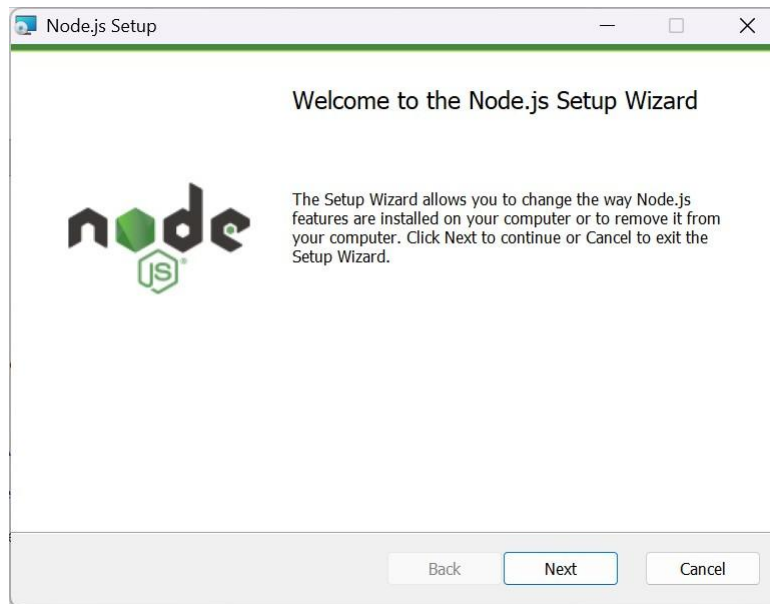
```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.

C:\xampp\htdocs\PPW_2311104060_Aulia Jasifa Br Ginting_SE-07-02\PERTEMUAN13\Praktikum>notepad helloworld.js
C:\xampp\htdocs\PPW_2311104060_Aulia Jasifa Br Ginting_SE-07-02\PERTEMUAN13\Praktikum>node helloworld.js
Server running at http://127.0.0.1:3000/
```



b. Inisialisasi Proyek Node.js

Install node.js

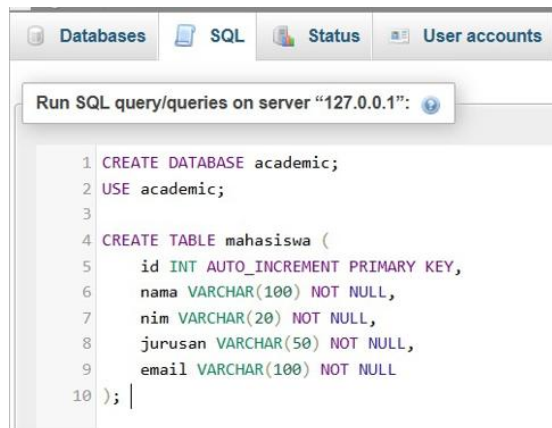


Cek node.js sudah terpasang

```
C:\Users\LENOVO>node -v  
v24.12.0  
  
C:\Users\LENOVO>npm -v  
11.6.2
```

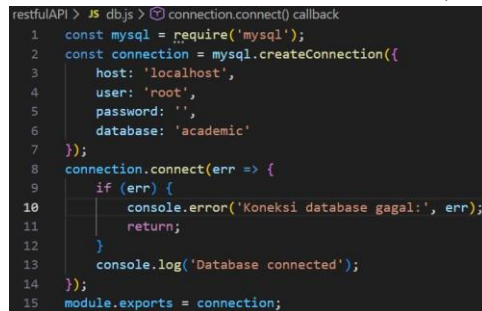
c. Konfigrasi Database

1. Membuat Database dan Tabel



```
1 CREATE DATABASE academic;
2 USE academic;
3
4 CREATE TABLE mahasiswa (
5     id INT AUTO_INCREMENT PRIMARY KEY,
6     nama VARCHAR(100) NOT NULL,
7     nim VARCHAR(20) NOT NULL,
8     jurusan VARCHAR(50) NOT NULL,
9     email VARCHAR(100) NOT NULL
10 );
```

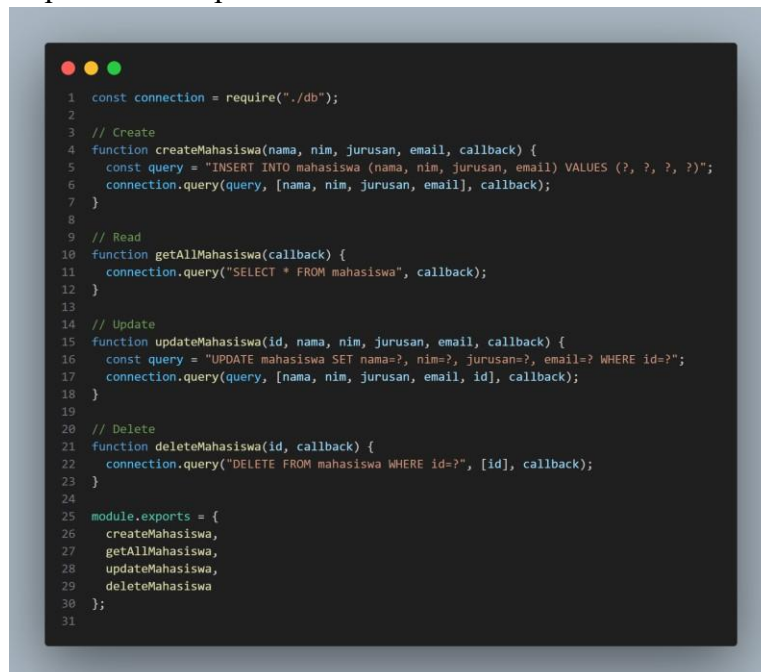
2. Membuat File Koneksi Database (db.js)



```
restfulAPI > JS db.js > connection.connect() callback
1 const mysql = require('mysql');
2 const connection = mysql.createConnection({
3     host: 'localhost',
4     user: 'root',
5     password: '',
6     database: 'academic'
7 });
8 connection.connect(err => {
9     if (err) {
10         console.error('Koneksi database gagal:', err);
11         return;
12     }
13     console.log('Database connected');
14 });
15 module.exports = connection;
```

d. Restful API

1. Implementasi Operasi CRUD



```
1 const connection = require("./db");
2
3 // Create
4 function createMahasiswa(nama, nim, jurusan, email, callback) {
5     const query = "INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES (?, ?, ?, ?)";
6     connection.query(query, [nama, nim, jurusan, email], callback);
7 }
8
9 // Read
10 function getAllMahasiswa(callback) {
11     connection.query("SELECT * FROM mahasiswa", callback);
12 }
13
14 // Update
15 function updateMahasiswa(id, nama, nim, jurusan, email, callback) {
16     const query = "UPDATE mahasiswa SET nama=?, nim=?, jurusan=?, email=? WHERE id=?";
17     connection.query(query, [nama, nim, jurusan, email, id], callback);
18 }
19
20 // Delete
21 function deleteMahasiswa(id, callback) {
22     connection.query("DELETE FROM mahasiswa WHERE id=?", [id], callback);
23 }
24
25 module.exports = {
26     createMahasiswa,
27     getAllMahasiswa,
28     updateMahasiswa,
29     deleteMahasiswa
30 };
31
```

2. Pembuatan File Utama Aplikasi (app.js)

```
1  const express = require("express");
2  const dbOperations = require("./crud");
3
4  const app = express();
5  const port = 3000;
6
7  app.use(express.json());
8
9  // CREATE
10 app.post("/mahasiswaCreate", (req, res) => {
11   const { nama, nim, jurusan, email } = req.body;
12   dbOperations.createMahasiswa(nama, nim, jurusan, email, err => {
13     if (err) return res.status(500).send("Error creating");
14     res.send("Mahasiswa created");
15   });
16 });
17
18 // READ
19 app.get("/mahasiswaGet", (req, res) => {
20   dbOperations.getAllMahasiswa(err, results) => {
21     if (err) return res.status(500).send("Error fetching");
22     res.json(results);
23   };
24 });
25
26 // UPDATE
27 app.put("/mahasiswaUpdate/:id", (req, res) => {
28   const { id } = req.params;
29   const { nama, nim, jurusan, email } = req.body;
30   dbOperations.updateMahasiswa(id, nama, nim, jurusan, email, err => {
31     if (err) return res.status(500).send("Error updating");
32     res.send("Mahasiswa updated");
33   });
34 });
35
36 // DELETE
37 app.delete("/mahasiswaDelete/:id", (req, res) => {
38   dbOperations.deleteMahasiswa(req.params.id, err => {
39     if (err) return res.status(500).send("Error deleting");
40     res.send("Mahasiswa deleted");
41   });
42 });
43
44 app.listen(port, () => {
45   console.log("Server running at http://localhost:${port}");
46 });
```

▼ PRAKTIKUM

▼ restfulAPI

> node_modules

JS app.js

JS crud.js

JS db.js

{ } package-lock.json

{ } package.json

struktur proyek node.js

Ketika server berhasil dijalankan, Node.js akan menampilkan pesan pada terminal

B. UNGUIDED (Tugas Mandiri)

1. Soal 1:

Mahasiswa diminta untuk membangun sebuah aplikasi web sederhana untuk pengelolaan data mahasiswa berbasis Node.js, Express.js, dan MySQL.

Aplikasi harus menyediakan RESTful API serta dapat diakses melalui browser menggunakan halaman web sederhana (HTML dan JavaScript).

Mahasiswa diperbolehkan menggunakan proyek yang telah dibuat di atas atau membuat proyek baru.

Jawaban:

Class db.js

```
restfulAPI > JS db.js > ...
1  const mysql = require("mysql");
2
3  const connection = mysql.createConnection({
4    host: "localhost",
5    user: "root",
6    password: "",
7    database: "pendidikan",
8  });
9
10 connection.connect(err => {
11   if (err) {
12     console.error("Koneksi database gagal", err);
13     return;
14   }
15   console.log("Database connected");
16 });
17
18 module.exports = connection;
```

Class crud.js

```
1  const connection = require("./db");
2  // Create
3  function createMahasiswa(nama, nim, jurusan, email, callback) {
4      const query =
5          "INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES (?, ?, ?, ?)";
6      connection.query(query, [nama, nim, jurusan, email], (error,
7          results) => {
8          if (error) {
9              return callback(error, null);
10         }
11         callback(null, results);
12     });
13 }
14
15 // Read
16 function getAllMahasiswa(callback) {
17     const query = "SELECT * FROM mahasiswa";
18     connection.query(query, (error, results) => {
19         if (error) {
20             return callback(error, null);
21         }
22         callback(null, results);
23     });
24 }
25
26 // Update
27 function updateMahasiswa(id, nama, nim, jurusan, email, callback) {
28     const query =
29         "UPDATE mahasiswa SET nama = ?, nim = ?, jurusan = ?, email = ? WHERE id = ? ";
30     connection.query(query, [nama, nim, jurusan, email, id], (error,
31         results) => {
32         if (error) {
33             return callback(error, null);
34         }
35         if (results.affectedRows === 0) {
36             return callback(new Error("No rows updated, ID may not exist"),
37                 null);
38         }
39         callback(null, results);
40     });
41 }
42
43 // Delete
44 function deleteMahasiswa(id, callback) {
45     const query = "DELETE FROM mahasiswa WHERE id = ?";
46     connection.query(query, [id], (error, results) => {
47         if (error) {
48             return callback(error, null);
49         }
50         callback(null, results);
51     });
52 }
53
54 module.exports = {
55     getAllMahasiswa,
56     createMahasiswa,
57     updateMahasiswa,
58     deleteMahasiswa,
59 };
```


Class app.js

```
1  const express = require("express");
2  const mysql = require("mysql");
3  const app = express();
4
5  app.use(express.json());
6  app.use(express.static("public"));
7
8  const db = mysql.createConnection({
9    host: "localhost",
10   user: "root",
11   password: "",
12   database: "pendidikan"
13 });
14
15 db.connect(err => {
16   if (err) throw err;
17   console.log("MySQL Connected");
18 });
19
20 // GET
21 app.get("/api/mahasiswa", (req, res) => {
22   db.query("SELECT * FROM mahasiswa", (err, result) => {
23     if (err) return res.status(500).send(err);
24     res.json(result);
25   });
26 });
27
28 // POST
29 app.post("/api/mahasiswa", (req, res) => {
30   const { nama, nim, jurusan, email } = req.body;
31
32   const sql = "INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES (?, ?, ?, ?)";
33   db.query(sql, [nama, nim, jurusan, email], (err) => {
34     if (err) return res.status(500).send(err);
35     res.send("Data berhasil ditambahkan");
36   });
37 });
38
39 // PUT
40 app.put("/api/mahasiswa/:id", (req, res) => {
41   const { id } = req.params;
42   const { nama, nim, jurusan, email } = req.body;
43
44   const sql = `
45     UPDATE mahasiswa
46     SET nama=?, nim=?, jurusan=?, email=?
47     WHERE id=?
48   `;
49
50   db.query(sql, [nama, nim, jurusan, email, id], (err, result) => {
51     if (err) return res.status(500).send(err);
52
53     if (result.affectedRows === 0) {
54       return res.status(404).send("Data tidak ditemukan");
55     }
56
57     res.send("Data berhasil diupdate");
58   });
59 });
60
61 // DELETE
62 app.delete("/api/mahasiswa/:id", (req, res) => {
63   const { id } = req.params;
64
65   db.query("DELETE FROM mahasiswa WHERE id=?", [id], (err, result) => {
66     if (err) return res.status(500).send(err);
67
68     if (result.affectedRows === 0) {
69       return res.status(404).send("Data tidak ditemukan");
70     }
71
72     res.send("Data berhasil dihapus");
73   });
74 });
75
76 const port = 3000;
77
78 app.listen(port, () => {
79   console.log(`Server berjalan di http://localhost:${port}`);
80 });
```

```

1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4 <meta charset="UTF-8">
5 <title>Data Mahasiswa</title>
6
7 <!-- Bootstrap CSS -->
8 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
9
10 <style>
11 body {
12   background-color: #f4f6f8;
13 }
14 <card {
15   box-shadow: 0 4px 8px rgba(0,0,0,0.05);
16 }
17 </style>
18 </head>
19 <body>
20
21 <div class="container mt-4">
22   <h2 class="text-center mb-4">Aplikasi Data Mahasiswa</h2>
23
24   <!-- FORM TAMBAH -->
25   <div class="card mb-4">
26     <div class="card-body">
27       <h3 class="mb-3">Tambah Mahasiswa</h3>
28
29       <input id="nama" class="form-control mb-2" placeholder="Nama">
30       <input id="nim" class="form-control mb-2" placeholder="NIM">
31       <input id="jurusan" class="form-control mb-2" placeholder="Jurusan">
32       <input id="email" class="form-control mb-3" placeholder="Email">
33
34       <button class="btn btn-primary w-100" onclick="tambah()">Tambah Data</button>
35     </div>
36   </div>
37
38   <!-- LIST DATA -->
39   <div class="card">
40     <div class="card-body">
41       <h3 class="mb-3">Daftar Mahasiswa</h3>
42       <ul class="list-group" id="list"></ul>
43     </div>
44   </div>
45 </div>
46
47 <!-- Bootstrap JS -->
48 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
49
50 <script>
51 // GET
52 function loadData() {
53   fetch("/api/mahasiswa")
54     .then(res => res.json())
55     .then(data => {
56       const list = document.getElementById("list");
57       list.innerHTML = "";
58
59       data.forEach(e => {
60         list.innerHTML += `
61         <li class="list-group-item">
62           <div>{e.nama}</div>
63           NIM {e.nim}<br>
64           Jurusan {e.jurusan}<br>
65           Email: {e.email}<br>
66           <button class="btn btn-danger btn-sm mt-2" onclick="hapus({e.id})">Hapus</button>
67         </li>
68       `;
69     });
70   });
71
72 // POST
73 function tambah() {
74   fetch("/api/mahasiswa", {
75     method: "POST",
76     headers: { "Content-Type": "application/json" },
77     body: JSON.stringify({
78       nama: nama.value,
79       nim: nim.value,
80       jurusan: jurusan.value,
81       email: email.value
82     })
83   }).then(() => {
84     document.querySelectorAll("input").forEach(i => i.value = "");
85     loadData();
86   });
87
88 // DELETE
89 function hapus(id) {
90   if (confirm("Yakin menghapus data ini?")) {
91     fetch("/api/mahasiswa/{id}", { method: "DELETE" })
92       .then(() => loadData());
93   }
94 }
95
96 // PUT
97 function updateData() {
98   const id = editId.value;
99
100   fetch("/api/mahasiswa/{id}", {
101     method: "PUT",
102     headers: { "Content-Type": "application/json" },
103     body: JSON.stringify({
104       nama: editNama.value,
105       nim: editNim.value,
106       jurusan: editJurusan.value,
107       email: editEmail.value
108     })
109   }).then(() => {
110     editModal.hide();
111     loadData();
112   });
113 }
114
115 </script>
116
117 </body>
118 </html>

```

Output:

Tampilan awal dari web

The screenshot shows a web browser window with the title 'Aplikasi Data Mahasiswa'. The page has a light gray background. At the top center, there is a header with a small icon and the text 'Aplikasi Data Mahasiswa'. Below the header, there is a form with four input fields labeled 'Nama', 'NIM', 'Jurusan', and 'Email'. At the bottom of the form is a blue button labeled 'Tambah'.

Data mahasiswa

MODUL 13 - NODE JS.pdf

localhost / 127.0.0.1 / pendidikan

Data Mahasiswa

localhost:3000

Tambah Data

Daftar Mahasiswa

Berlian Seva

NIM: 2311104067

Jurusan: Software Engineering

Email: berlian@gmail.com

Hapus

Naura Alsha

NIM: 2311104078

Jurusan: Software Engineering

Email: naura@gmail.com

Hapus

Aulia Jasifa

NIM: 2311104060

Jurusan: Software Engineering

Email: zsfa@gmail.com

Hapus

Alya Rabani

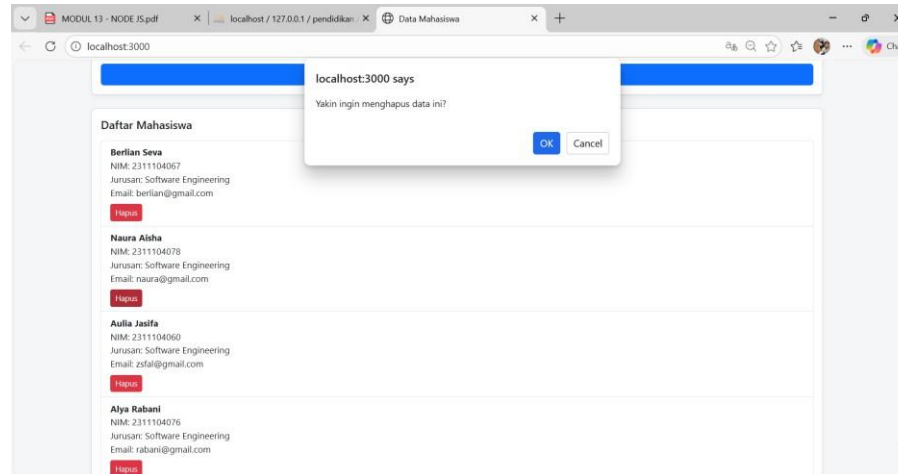
NIM: 2311104076

Jurusan: Software Engineering

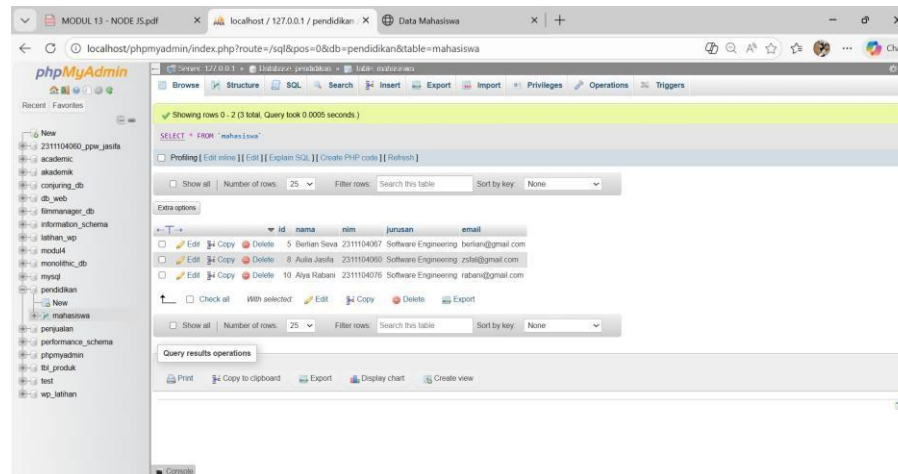
Email: rabani@gmail.com

Hapus

Menghapus salah satu data mahasiswa



Data tersimpan



Penjelasan:

Aplikasi yang dikembangkan pada tugas unguided ini merupakan aplikasi web sederhana untuk pengelolaan data mahasiswa berbasis Node.js, Express.js, dan MySQL dengan konsep RESTful API. Node.js digunakan sebagai runtime JavaScript di sisi server, sedangkan Express.js berperan sebagai framework untuk menangani routing dan permintaan HTTP. MySQL digunakan sebagai media penyimpanan data mahasiswa.

Aplikasi menyediakan empat endpoint utama sesuai konsep CRUD, yaitu GET untuk menampilkan seluruh data mahasiswa dari database, POST untuk menambahkan data mahasiswa baru, PUT untuk memperbarui data mahasiswa berdasarkan ID, dan DELETE untuk menghapus data mahasiswa tertentu. Setiap

endpoint diimplementasikan menggunakan query SQL yang dijalankan melalui koneksi MySQL.

Pada sisi frontend, aplikasi menggunakan HTML, JavaScript, dan Bootstrap untuk menampilkan antarmuka pengguna. Data mahasiswa ditampilkan secara dinamis melalui pemanggilan API menggunakan fungsi `fetch()`. Proses edit data dilakukan melalui modal form sehingga pengguna dapat memperbarui seluruh data dalam satu tampilan, sedangkan penghapusan data dilakukan melalui tombol hapus yang memanggil endpoint DELETE. Dengan mekanisme ini, aplikasi mampu mengelola data mahasiswa secara dinamis dan terintegrasi antara frontend dan backend.

BAB III

PENUTUP

A. Kesimpulan

Berdasarkan hasil praktikum dan tugas mandiri (unguided), dapat disimpulkan bahwa penggunaan Node.js dan Express.js memungkinkan pengembangan aplikasi web berbasis server-side JavaScript secara efisien. Penerapan konsep RESTful API dengan metode GET, POST, PUT, dan DELETE memudahkan proses pengelolaan data mahasiswa secara terstruktur sesuai konsep CRUD. Selain itu, integrasi dengan MySQL sebagai database mampu menyimpan dan mengelola data secara persisten.

Selama proses pengerjaan, beberapa kendala yang dihadapi antara lain kesalahan konfigurasi server, ketidaksesuaian antara parameter endpoint dengan primary key pada database, serta error pada proses update dan delete data. Kendala tersebut dapat diatasi dengan melakukan pengecekan struktur tabel database, memastikan penggunaan ID sebagai primary key, serta melakukan debugging pada route Express dan query SQL. Dari kegiatan ini, mahasiswa memperoleh pemahaman yang lebih baik mengenai alur kerja aplikasi web dari sisi frontend hingga backend serta pentingnya sinkronisasi antara antarmuka pengguna dan database.